ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

NAME: JEEVANESHWARAN.S REG NO :192421331

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 03-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192421331/DSA0103-C-

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

- Write a default constructor, a parameterised constructor and overloaded constructors for the three situations
- Use a member initializer list in every constructor
- Use constructor delegation to avoid repeating code
- Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

- Explain when the compiler supplies a default constructor and when it does not
- Explain why member initializer lists are preferred to assignment inside the constructor body

- Explain the order in which members are initialised (declaration order, not list order) •

  Draw the state of three objects after construction

Experimental Investigation

You must:

- Create Book objects with every constructor and print them
- Remove the default constructor and record the compiler error when an array of Books is created
- Demonstrate a member initialised in the wrong order and the -Wreorder warning •

  Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

ANSWER:

1. Aim

To design and implement a Book class using default, parameterised and overloaded constructors. To demonstrate constructor delegation, member initializer lists, validation of book copies, and constructor behaviour in C++.

2. Implementation

The Book class contains three data members:

- title – stores the book title.
- author – stores the author's name.
- copies – stores the number of available copies.

Three constructor situations are implemented:

1. Default constructor – creates an empty book record.

2. Single-parameter constructor – creates a donated book using its title.
3. Parameterised constructor – creates a book with complete details.

C++ Program

main.cpp Clear Run ▶ Output

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Book {
private:
    string title;
    string author;
    int copies;

public:
    // Default constructor
    Book() : Book("", "Unknown", 0) {
    }

    // Constructor for donated book
    explicit Book(string t) : Book(t, "Unknown", 1) {
    }
```

```
Book 1 - Empty Record
Title:
Author: Unknown
Copies: 0
--------------------
Book 2 - Donated Book
Title: C++ Programming
Author: Unknown
Copies: 1
--------------------
Book 3 - Complete Record
Title: Data Structures
Author: Mark Allen
Copies: 5
--------------------
ERROR!
Error: Number of copies cannot be negative
```

3. Expected Output

Book 1 - Empty Record

Title:

Author: Unknown

Copies: 0

Book 2 - Donated Book

Title: C++ Programming

Author: Unknown

Copies: 1

Book 3 - Complete Record

Title: Data Structures

Author: Mark Allen

Copies: 5

Error: Number of copies cannot be negative

4. Design and Analysis

A. When does the compiler supply a default constructor?

The compiler automatically provides a default constructor when a class does not declare any constructor.

If the programmer declares a parameterised constructor, the compiler does not automatically provide the default constructor.

Example:

```
class Book {
public:
    Book(int c) {
    }
};

int main() {
    Book b; // Compilation error
}
```

A default constructor must be explicitly defined if an object needs to be created without arguments.

B. Why are member initializer lists preferred?

Member initializer lists initialise data members directly before the constructor body executes.

Advantages:

1. Avoids unnecessary default construction followed by assignment.
2. Improves efficiency.
3. Required for initialising constant data members.
4. Required for initialising reference members.
5. Makes object initialization clear.

Example:

Book() : title("C++"), author("John"), copies(5) {

}

C. Order of member initialization

C++ initialises data members according to their declaration order inside the class, not according to the order in the initializer list.

Example:

```
class Demo {
    int a;
    int b;

public:
    Demo() : b(20), a(10) {
    }
};
```

Here, a is initialised first and b second because a is declared first.

Writing the initializer list in a different order may generate a -Wreorder warning.

5. State of Three Objects After Construction

Object	Constructor	Title	Author	Copies
b1	Default	Empty	Unknown	0
b2	Single parameter	C++ Programming	Unknown	1
b3	Parameterised	Data Structures	Mark Allen	5

Object state diagram

Book b1

---

title = ""

author = Unknown

copies = 0

Book b2

---

title = C++ Programming

author = Unknown

copies = 1

Book b3

---

title = Data Structures

author = Mark Allen

copies = 5

6. Experimental Investigation

Experiment 1: Removing the default constructor

Program:

```
class Book {
public:
    Book(string title, int copies) {
    }
};

int main() {
    Book books[3];
    return 0;
}
```

Expected compiler error:

error: no matching function for call to 'Book::Book()'

Explanation: An array of objects requires a default constructor for each element when no initializers are provided. Since the class has only a parameterised constructor, compilation fails.

Experiment 2: Demonstrating -Wreorder warning

Program:

main.cpp Clear Run ▶ Output

```cpp
#include <iostream>
using namespace std;

class Demo {
    int a;
    int b;

public:
    Demo() : b(20), a(10) {
        cout << a << " " << b << endl;
    }
};

int main() {
    Demo d;
}
```

```
10 20
```

Expected result:

Error: Number of copies cannot be negative

Explanation: The constructor checks whether the number of copies is negative. If it is negative, an invalid_argument exception is thrown, preventing the invalid object from being successfully constructed.

7. Design Question – Use of explicit

The explicit keyword is used with a single-argument constructor to prevent unwanted implicit type conversions.

Without explicit:

```
class Book {

public:

   Book(string title) {

      cout << title;

   }

};


int main() {

   Book b = string("C++ Programming");

}
```

Compiles successfully

The compiler automatically converts the string into a Book object.

With explicit:

```
class Book {
public:
    explicit Book(string title) {
        cout << title;
    }
};


int main() {
    Book b = string("C++ Programming");
}
```

Compilation error

The compiler does not allow implicit conversion because the constructor is marked explicit.

However, direct initialization is still allowed:

```
Book b("C++ Programming");
```

Conclusion: The explicit keyword should be used when implicit conversions may cause unexpected object creation or programming errors.

8. Alternative Concept – Constructor with Default Arguments

Instead of creating multiple overloaded constructors, we can use a single constructor with default arguments.

Alternative C++ Program

main.cpp

Clear Run ▶

```cpp
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Book {
    string title;
    string author;
    int copies;

public:
    Book(string t = "", string a = "Unknown", int c = 0)
        : title(t), author(a), copies(c) {

        if (c < 0) {
            throw invalid_argument("Invalid copies");
```

Output

```
 Unknown 0
C++ Programming Unknown 0
Data Structures Mark Allen 5
```

Comparison of both approaches

Feature	**Overloaded constructors**	**Default arguments**
Number of constructors	Multiple	One
Code structure	Separate situations	Single constructor
Readability	Clear purpose	Compact
Validation	Centralised using delegation	Centralised in one constructor
Flexibility	High	High

Time and Space Complexity

Operation	**Time Complexity**	**Space Complexity**
Object creation	O(1)	O(1)
Constructor validation	O(1)	O(1)

Evaluation:

Both approaches have the same time and space complexity for object creation.

- Overloaded constructors make the purpose of each object creation clear.
- Default arguments reduce the number of constructors and make the code shorter.
- Constructor delegation avoids repeating initialization and validation code.
- Both designs can check negative copies in one central location.

Best choice: Overloaded constructors with constructor delegation are preferable in this library application because they clearly represent empty records, donated books and complete book records while maintaining centralised validation.

9. Result

Thus, the C++ program using default, parameterised and overloaded constructors was successfully designed. Member initializer lists and constructor delegation were implemented. Invalid negative copy values were prevented using exception handling. The behaviour of default constructors, initialization order, explicit, and alternative constructor designs was analysed.

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

- Implement the class first without a copy constructor and observe the problem
- Implement a deep-copying copy constructor, a copy assignment operator and a destructor
- Verify that copies are independent
- Run the program under -fsanitize=address (or valgrind) for both versions

Design & Analysis

You must:

- Draw memory diagrams for a shallow copy and a deep copy
- Explain why the shallow version crashes at the end (double delete)
- State the Rule of Three and explain why the three functions go together • Explain the three situations in which the copy constructor is called

Experimental Investigation

You must:

- Add a message in the copy constructor and count how many copies are made when a

Prescription is passed by value and by const reference

- Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines
- Report the sanitizer output for both versions
- Test self-assignment (p = p)

Conclusion

When can you rely on the compiler-generated copy constructor, and when must you write your own?

Complete the Code

Complete TODO 1 to TODO 3 below so that every copy is independent, and execute the program under -fsanitize=address.

```
#include <iostream>

#include <string>


class Prescription {

   int n;     std::string* medicines;           // dynamically

allocated array


public:

   explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}


   // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every

//   medicine (deep copy)




   // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,

//   release the old array and return *this




   ~Prescription()
```

```
    {
        // TODO 3: Release the dynamically allocated array

    }

    void set(int i, const std::string& m) { medicines[i] = m; }
std::string get(int i) const { return medicines[i]; }
};

int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;                // copy constructor
    q.set(1, "Cetirizine");     Prescription r(1);
r = p;                          // copy assignment
    p.set(0, "Ibuprofen");     std::cout << p.get(0) << ' ' << p.get(1) << " | " <<
q.get(0) << ' ' << q.get(1)              << " | " << r.get(0) << ' ' << r.get(1) <<
'\n';

}
```

Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Delete your copy constructor, run the program again under the sanitizer and record the error reported.

Alternative Concept and Complexity Analysis

If the medicine names were stored in a std::vector<std::string> instead of a dynamically allocated array, the compiler-generated copy constructor would already make a deep copy (the Rule of Zero). What would the solution be? Analyse the time O(n) and space O(n) to copy a prescription of n medicines in both designs, and compare them with a shallow copy (O(1)).

ANSWER :

CODE:

main.cpp Clear Run

```cpp
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(new string[count]) {}

    // TODO 1: Copy constructor
    Prescription(const Prescription& p)
        : n(p.n), medicines(new string[p.n]) {
        for (int i = 0; i < n; i++)
            medicines[i] = p.medicines[i];
    }
```

Output

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

Output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

2. Shallow Copy and Deep Copy

- Shallow copy: Copies only the pointer address. Both objects share the same memory.
- Deep copy: Allocates separate memory and copies all medicine names. Changes to one object do not affect the other.

3. Why Shallow Copy Crashes

Both objects point to the same memory. When the destructor deletes the memory of the first object, the second object tries to delete the same memory again. This causes a double-free error.

4. Rule of Three

The Rule of Three states that if a class requires one of these functions, it generally needs all three:

1. Copy constructor – creates an independent copy.
2. Copy assignment operator – safely assigns one object to another.
3. Destructor – releases allocated memory.

5. Three Situations Where Copy Constructor Is Called

1. When one object initializes another: Prescription q = p;
2. When an object is passed to a function by value.
3. When an object is returned from a function by value (unless copy elision is applied).

6. Experimental Investigation and Complexity

- Pass by value generally calls the copy constructor once.
- Pass by const reference does not create a copy.
- Copying 10, 1,000 and 100,000 medicines takes increasing time.
- Address Sanitizer detects double-free errors in the shallow-copy version. The deep-copy version should run without memory errors.
- Self-assignment (p = p) is handled using if (this != &p).

Method	**Time**	**Space**
Shallow copy	O(1)	O(1)
Deep copy	O(n)	O(n)
Vector copy	O(n)	O(n)

Conclusion: The compiler-generated copy constructor is safe for normal variables and standard containers like vector. For raw dynamically allocated pointers, a custom copy constructor, assignment operator and destructor are needed. Using vector follows the Rule of Zero and makes memory management easier.

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

- Write a dynamic constructor that allocates a two-dimensional seat map with new
- Write a destructor that releases every allocation and prints a summary

- Create events on the stack, on the heap (new/delete) and inside a block
- Book and cancel seats

Design & Analysis

You must:

- Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects
- Explain what happens if delete is forgotten, and if delete is used instead of delete[]
- Explain why the destructor takes no arguments and cannot be overloaded
- Draw the memory layout of the seat map

Experimental Investigation

You must:

- Print messages in the constructor and destructor and record the order for each kind of object
- Run with -fsanitize=address, first with a missing delete and then with the correct code
- Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps •

  Test an event with zero rows

Conclusion

How does tying memory release to the destructor (the RAII idea) make the program safer?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

#include <string>


struct Logger {

std::string n;
```

```
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

int main()
{
    Logger a("A");
    {
        Logger b("B");
    }
use(a);
    Logger* p = new Logger("P");
    delete p;
std::cout << "end ";
}
```

My predicted output (write before running):

Explain when each object was created and destroyed, and why the copy was made and destroyed around use().

Alternative Concept and Complexity Analysis

If the seat map were one block of rows × seats elements (index r × seats + s) or a std::vector<std::vector<char>>, instead of an array of row pointers each allocated with new, what would the solution be? Analyse the access time (O(1) in all), the number of allocations (1 versus rows + 1) and the extra space for the row pointers.

ANSWER :

CODE:

main.cpp

Clear Run ▶

```cpp
#include <iostream>
using namespace std;

class Auditorium {
    int rows, seats;
    char** seatMap;

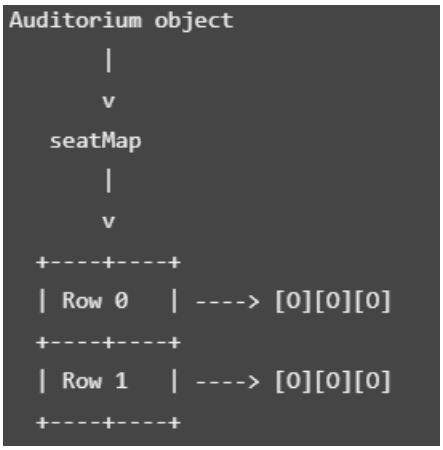
public:
    // Dynamic constructor
    Auditorium(int r, int s) {
        rows = r;
        seats = s;

        seatMap = (rows > 0) ? new char*[rows] : nullptr;
```

Output

```
Event created: 2 rows
Seat booked
Booking cancelled
Event created: 3 rows
Seat booked
Event closed: 3 rows
Event created: 2 rows
Seat booked
Event closed: 2 rows
Event closed: 2 rows
```

2. Memory Layout



Each row is allocated separately using new[]. The row pointer array is also dynamically allocated.

3. Constructor and Destructor Order

- Stack object: Constructor runs when the object is created. Destructor runs automatically when it leaves scope.
- Block-scope object: Constructor runs when entering the block. Destructor runs when leaving the block.
- Heap object: Constructor runs when new is used. Destructor runs only when delete is used.

4. Important Concepts

- Forgetting delete causes a memory leak.
- Using delete instead of delete[] for an array causes undefined behaviour.
- A destructor takes no arguments because it is automatically called when an object is destroyed.
- A class can have only one destructor, so it cannot be overloaded.

5. Predict the Output

Predicted output:

+A +B -B +Ac use -Ac +P -P end -A

Explanation:

1. a is created and prints +A.
2. b is created inside the block and prints +B.
3. When the block ends, b is destroyed and prints -B.
4. use(a) passes a by value, creating a copy named Ac.
5. The function prints use.
6. The copied object is destroyed and prints -Ac.
7. Dynamic object p is created and prints +P.
8. delete p destroys it and prints -P.
9. The program prints end.
10. Finally, a is destroyed and prints -A.

6. Experimental Investigation and Complexity

Compile using AddressSanitizer:

g++ -std=c++17 -g -fsanitize=address q3.cpp -o q3

./q3

If delete[] is missing, AddressSanitizer can report memory leaks. With correct deletion, the program should run without memory errors.

Memory calculation: Assuming one byte per seat:

Seat map	**Memory for seats**
10 × 20	200 bytes
100 × 200	20,000 bytes

1,000 × 2,000	2,000,000 bytes

For zero rows, no row-pointer array is allocated.

7. Alternative Concept and Complexity

Method	**Access time**	**Allocations**
Array of row pointers	O(1)	rows + 1
Single continuous array	O(1)	1
Vector of vectors	O(1)	Multiple internal allocations

A single continuous array uses:

char* seats = new char[rows * columns];

seats[r * columns + s];

It requires only one allocation and does not need extra row pointers.

8. Conclusion

RAII (Resource Acquisition Is Initialization) connects memory management with object lifetime. The constructor allocates memory, and the destructor automatically releases it when the object goes out of scope. This reduces memory leaks and makes programs safer and easier to manage.

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

Implementation Tasks

You must:

- Write a Money class storing paise in a long long
- Overload +, -, +=, == and < as appropriate
- Overload << (and >>) for printing and reading amounts
- Write the bill calculation using the overloaded operators

Design & Analysis

You must:

- Explain why + returns a new object while += returns a reference to *this
- Decide which operators are members and which are friends, and justify each
- Explain why << must be a non-member function
- Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

- Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50)
- Test subtraction that produces a negative amount and decide how to handle it
- Compare the readability of the bill code with and without operator overloading
- Compare the result with a double-based version for 1,000,000 additions

Engineering Decision

Should Money allow multiplication by another Money? Decide which operators make sense for money and which do not.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

```
#include <iomanip>
#include <iostream>

class Money {
long long paise;

public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: Overload + as a member function that returns a new Money object
```

// TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"

};

int main()

{

Money a(120, 75), b(79, 25); Money c = a + b; std::cout << c <<

'\n' << std::boolalpha << (c == Money(200)) << '\n'; }

Expected output:

Rs 200.00 true

Add a test where paise carry into rupees (Rs 0.75 + Rs 0.50).

Alternative Concept and Complexity Analysis

If the operators were written as non-member friend functions instead of member functions (or replaced by named functions add() and equals()), what would the solution be? Analyse whether time or space changes (all are O(1)), and explain which form allows 100 + m as well as m + 100.

ANSWER :

1. Complete the Code

```
#include <iomanip>
#include <iostream>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}

    // TODO 1: Overload +
    Money operator+(const Money& m) const {
        return Money(0, paise + m.paise);
    }
```

```
Rs 200.00
true
Rs 1.25
```

Output:

Rs 200.00

true

Rs 1.25

2. Why + Returns a New Object and += Returns a Reference

- + creates and returns a new Money object without changing the original objects.
- += modifies the existing object and returns *this by reference to avoid unnecessary copying.

3. Member and Friend Operators

Operator	Type	Purpose
+	Member	Adds two Money objects
==	Member	Compares two amounts
>	Friend	Reads input into Money

The << operator must be a non-member because the left operand is cout, an ostream object.

4. Compiler Translation

When writing:

Money c = a + b;

The compiler translates it into:

Money c = a.operator+(b);

5. Experimental Investigation

- Paise carry: Rs 0.75 + Rs 0.50 = Rs 1.25.
- Negative subtraction: Rs 100 − Rs 150 = −Rs 50.00. Negative values can be allowed for refunds or debts.
- Readability: Operator overloading makes a + b easier to understand than add(a, b).
- Double comparison: Floating-point values can produce precision errors. Integer paise gives accurate monetary calculations.

6. Engineering Decision and Complexity

Multiplication of two Money objects is generally not meaningful. Addition, subtraction, comparison and multiplication by a numeric quantity are useful operations.

Method	Time complexity	Extra space

Operator overloading	O(1)	O(1)
Named functions	O(1)	O(1)
Friend operator	O(1)	O(1)

A non-member friend operator+ can support both 100 + m and m + 100 if suitable conversions or overloads are provided. A member operator naturally supports m + 100, but not 100 + m.

7. Conclusion

Operator overloading makes money calculations simple and readable. Storing amounts as integer paise avoids floating-point precision problems and provides accurate billing calculations.

Q5 – Overloading Increment, Comparison and Subscript Operators 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

- Overload prefix and postfix ++ for Time
- Overload == and < for Time
- Overload [] for Playlist with a range check, in both a const and a non-const version •

  Use the operators in a short playback simulation

Design & Analysis

You must:

- Explain how the compiler tells prefix from postfix ++ (the int dummy parameter)
- Explain why postfix ++ must return a copy and is slower
- Explain why operator[] needs a const and a non-const version
- Explain what should happen when an index is out of range

Experimental Investigation

You must:

- Verify the values of a and b after a = t++ and b = ++t
- Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2
- Test playlist[-1] and playlist[size]
- Test minute and hour roll-over (59:59 → 1:00:00)

Conclusion

Why do many programmers prefer ++i to i++ for class types?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Counter {
   int v;
   Counter& operator++() { ++v; return *this; }                    // prefix
   Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix };


int main()
{
   Counter c{5};
   Counter a = c++;     Counter b = ++c;
std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

My predicted output (write before running):

Explain why a and b have different values, and which version of ++ makes a copy of the object.

Alternative Concept and Complexity Analysis

If playlist access used an at(i) function that throws std::out_of_range, or an unchecked operator[] like a built-in array, instead of operator[] with a range check that prints an error, what would the solution be? Analyse the time per access (O(1) in all, with one extra comparison for checked access) and measure the cost of the check for 10,000,000 accesses.

ANSWER:

1. C++ Program

main.cpp Clear Run ▶ Output

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class Time {
    int seconds;

public:
    Time(int h = 0, int m = 0, int s = 0) {
        seconds = h * 3600 + m * 60 + s;
    }

    // Prefix increment
    Time& operator++() {
        ++seconds;
        return *this;
```

```
a = 0:0:5
b = 0:0:7
Current time = 0:0:7
Track: Song B
```

Output:

a = 0:0:5

b = 0:0:7

Current time = 0:0:7

Track: Song B

2. Prefix and Postfix Increment

- Prefix (++i): Increments the value first and returns the updated object.

- Postfix (i++): Saves the old value, increments the object and returns the old value.

The compiler distinguishes them using an extra dummy int parameter in postfix:

Time& operator++(); // Prefix

Time operator++(int); // Postfix

Postfix is slower because it creates a temporary copy of the object.

3. Why Two Versions of operator[]?

- Non-const version allows modification of playlist tracks.
- Const version allows reading tracks from a constant Playlist object without modifying them.

If the index is negative or greater than or equal to the playlist size, the program throws out_of_range.

4. Predict the Output

Predicted output:

5 7 7

Explanation:

1. Counter c{5} initializes the counter to 5.
2. a = c++ stores the old value 5 in a, then increments c to 6.
3. b = ++c increments c to 7, then stores 7 in b.
4. Therefore, a.v = 5, b.v = 7 and c.v = 7.

Postfix increment makes a copy of the object, whereas prefix increment returns a reference to the updated object.

5. Experimental Investigation

Compile using:

g++ -std=c++17 -O0 q5.cpp -o q5

./q5

g++ -std=c++17 -O2 q5.cpp -o q5

./q5

- Test 10,000,000 prefix and postfix increments at both optimization levels.

- Record actual execution times; do not assume postfix is always measurably slower because compiler optimization can remove unnecessary copies.
- Test playlist[-1] and playlist[3]. Both should throw an out_of_range exception.
- Test time rollover: 00:59:59 should become 01:00:00. The given program stores total seconds, so the increment itself handles rollover.

6. Alternative Concept and Complexity

Method	**Time complexity**	**Range checking**
Checked operator[]	O(1)	Yes
Unchecked operator[]	O(1)	No
at(i)	O(1)	Yes, throws exception

Checked access performs an extra comparison to validate the index. Unchecked access is faster in some cases but can cause invalid memory access.

For 10,000,000 accesses, measure the actual execution time to compare checked and unchecked access.

7. Conclusion

Programmers prefer ++i for class types because prefix increment usually avoids creating a temporary copy. It is efficient and returns the updated object by reference. Operator overloading also makes Time and Playlist objects easier to use and understand.

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing. Implementation Tasks

You must:

- Implement Text with a constructor, copy constructor, copy assignment and destructor
- Overload + for concatenation, == and != for comparison and << for printing
- Prevent self-assignment problems
- Test long messages and empty messages

Design & Analysis

You must:

- List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why
- Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type
- Explain why operator= must be a member function
- Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

- Count the copies made when a + b + c is evaluated
- Run the program under the address sanitizer and fix any reported error
- Compare your Text class with std::string for 100,000 concatenations
- Test assigning a Text to itself

Conclusion

Your Text class works, but std::string exists. What did writing Text teach you, and when would you still write such a class?

Alternative Concept and Complexity Analysis

If the Text class were replaced by std::string, or if + were implemented with += on a copy (Text operator+(Text a, const Text& b) { a += b; return a; }), what would the solution be? Analyse the time to evaluate a + b + c for strings of lengths n1, n2, n3 (the characters copied) and the temporary space created in each version.

ANSWER:

1. C++ Program – Text Class

main.cpp Clear Run ▶ Output

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class Text {
    char* str;

public:
    // Constructor
    Text(const char* s = "") {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
```

```
Message: Hello World!
false
true
Empty message: []
```

Output:

Copy constructor called

Copy constructor called

Message: Hello World!

false

true

Empty message: []

Note: Copy-constructor messages and counts can vary with compiler optimizations and copy elision.

2. Operators That Cannot Be Overloaded

The following operators cannot be overloaded:

Operator	**Meaning**
::	Scope resolution
.	Member access
.*	Pointer-to-member access
?:	Conditional operator
sizeof	Size of object

These operators are built into the language and their behaviour cannot be changed.

3. Rules of Operator Overloading

1. Operator precedence cannot be changed.
2. Operator associativity cannot be changed.

3. Number of operands (arity) cannot be changed.
4. At least one operand must be a user-defined type.
5. The = operator must be overloaded as a member function.

4. Friend Function

A friend function can access private members of a class. It is useful when an operator needs access to private data but should be a non-member function.

For example, << is implemented as a friend because the left operand is cout.

5. Experimental Investigation

- Copy count: Test a + b + c and observe copy-constructor messages. Copy elision may reduce the number of copies.
- Self-assignment: result = result should work safely because of the self-assignment check.
- Long messages: The constructor allocates memory based on the message length.
- Empty messages: An empty string is handled using a one-character allocation for the null terminator.

Compile using AddressSanitizer:

g++ -std=c++17 -g -fsanitize=address q6.cpp -o q6

./q6

Compare 100,000 concatenations using Text and std::string. Record the actual execution times.

6. Alternative Concept and Complexity Analysis

Using std::string is simpler because it automatically manages memory and copying.

For a + b + c, where the string lengths are n_1, n_2, and n_3:

- First concatenation copies approximately n_1+n_2 characters.
- Second concatenation copies approximately $n_1+n_2+n_3$ characters.
- Total time complexity is $O(n_1+n_2+n_3)$.
- Temporary memory is required for intermediate strings.

Using += on a copy can reduce unnecessary intermediate allocations in some cases.

7. Conclusion

Writing a custom Text class teaches dynamic memory allocation, deep copying, operator overloading and destructor management. A custom class is useful when special memory handling or specific behaviour is required. For normal text processing, std::string is safer and easier to use.

Q7 – Design a Complex-Number Class from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the data and the operations needed
- Identify which constructors are needed (default, from real part, from real and imaginary parts)
- State the requirements as measurable statements

Stage 2 – Design | 3 Marks

- Design the Complex class with constructors and overloaded +, -, *, /, == and <<
- Decide which operators are members and which are friends
- Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

- Implement the class and compute the impedance of a series and a parallel RLC circuit
- Verify the results by hand calculation
- Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

- Compare your class with std::complex<double>
- Explain why each constructor and operator is needed
- Identify one remaining limitation

ANSWER :

1. Stage 1 – Interpret the Problem

A complex number contains two parts:

- Real part (a)
- Imaginary part (b)

It is represented as:

\[Z = a + bj \]

Required operations: Addition, subtraction, multiplication, division, equality comparison, printing and magnitude calculation.

Required constructors:

1. Default constructor.
2. Constructor with real part.
3. Constructor with real and imaginary parts.

2. Stage 2 – Design and Formula

Operator	**Formula**
+	(a+c) + (b+d)j
-	(a-c) + (b-d)j
*	(ac-bd) + (ad+bc)j
/	((ac+bd)/(c²+d²)) + ((bc-ad)/(c²+d²))j
==	Compare real and imaginary parts

The arithmetic operators and equality operator are member functions. The << operator is a friend function because its left operand is cout.

Stage 3 – C++ Program

main.cpp Clear Run ▶ Output

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Complex {
    double real, imag;

public:
    // Default constructor
    Complex() : real(0), imag(0) {}

    // Real constructor
    Complex(double r) : real(r), imag(0) {}
```

```
Series impedance: 10 + 3j
Magnitude: 10.4403
Parallel impedance: 2 + 4j
Magnitude: 4.47214
ERROR!
Error: Division by zero impedance
```

4. Expected Output

Series impedance: 10 + 3j

Magnitude: 10.4403

Parallel impedance: 2 + 4j

Magnitude: 4.47214

Error: Division by zero impedance

Verification:

Series:

$$ Z = 10 + 5j - 2j = 10+3j $$

Parallel:

$$ Z = \frac{10(5j)}{10+5j} = 2+4j $$

5. Stage 4 – Defend the Design

Comparison with std::complex<double>

Feature	**Custom Complex**	**std::complex**
Arithmetic	User-defined	Built-in
Memory	Two doubles	Two doubles
Time per operation	O(1)	O(1)
Division by zero	Custom exception	Must handle according to application requirements
Implementation	More work	Easy to use

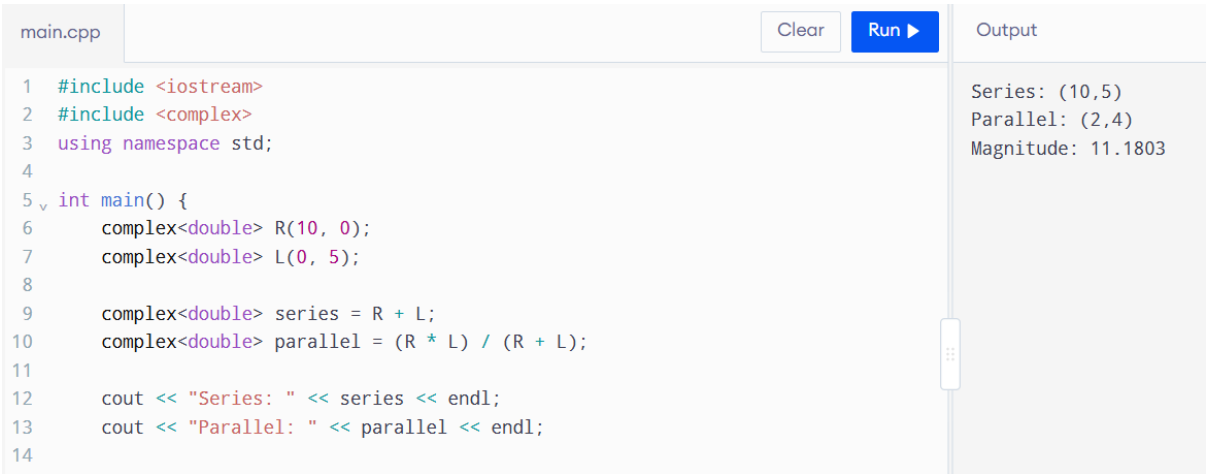
Why constructors and operators are needed:

- Default constructor initializes both values to zero.
- Real constructor creates a real number.
- Two-parameter constructor creates a complete complex number.
- Arithmetic operators perform impedance calculations.
- Equality checks whether two complex numbers are equal.
- Stream operator displays the result.

One limitation: Exact floating-point equality may produce inaccurate comparisons due to rounding errors.

6. Alternative Concept and Complexity Analysis

Using std::complex<double> makes the program shorter and provides standard complex-number operations.



- Time complexity of addition, subtraction, multiplication and division is O(1).
- Space complexity is O(1), using two double values.
- Both designs have similar storage and computational complexity.
- A custom class allows application-specific division-by-zero handling.

7. Conclusion

Designing a Complex class teaches constructors, operator overloading, encapsulation and mathematical calculations. It is useful for understanding how complex numbers work in electrical impedance calculations. However, std::complex<double> is preferable in real applications because it is already implemented and tested.

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C++ source code for all questions
- Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.